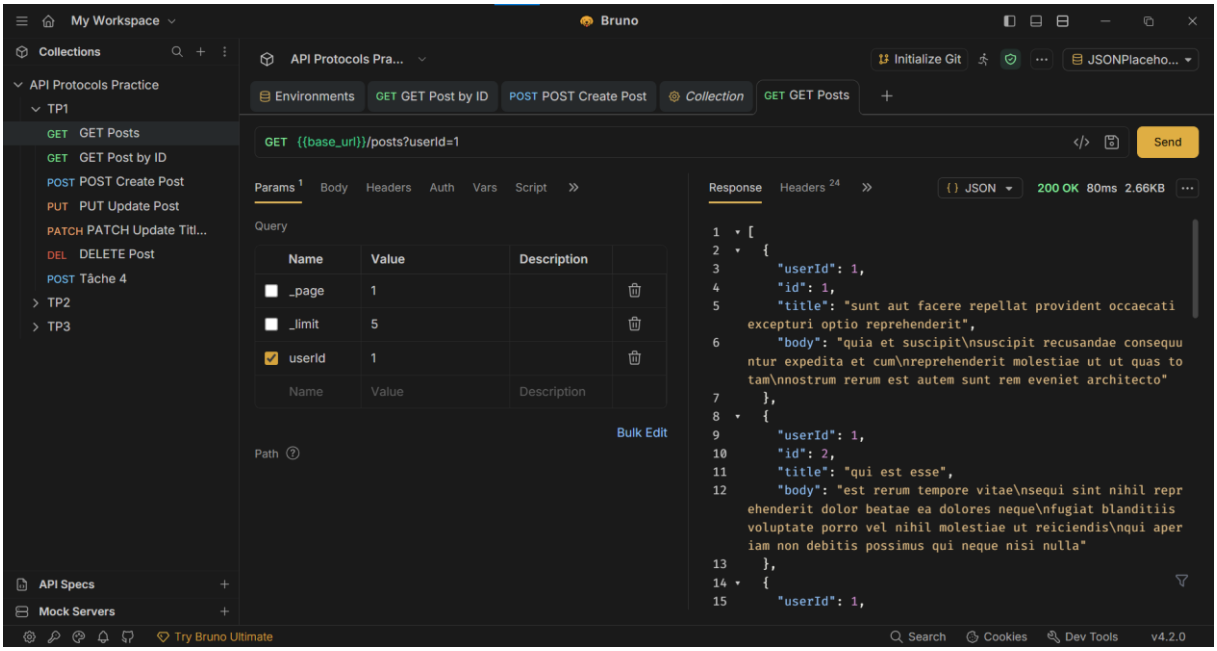
Rapport – TP1

Tache 1 :

- Quel est le code de statut renvoyé, et que signifie-t-il ?
- *Code 200, la récupération s'est bien déroulée*
- Combien d'éléments contient le tableau JSON renvoyé ?
- *100 éléments*
- Comment les données sont-elles structurées (clés, types de valeurs) ?
- *Les données sont structurées en JSON*
- combien de résultats obtenez-vous dans chaque cas ? En quoi cette approche (chaîne de requête) est-elle préférable à la création d'un endpoint dédié comme /posts/userId/1 ?
- *avec le paramètre userID=1 on obtient 5 éléments. Avec page et limit 5 éléments aussi. Cette approche est préférable car si on devait avoir /posts/userId/1/page/1/limit/5 ça deviendrait vite complexe à comprendre.*

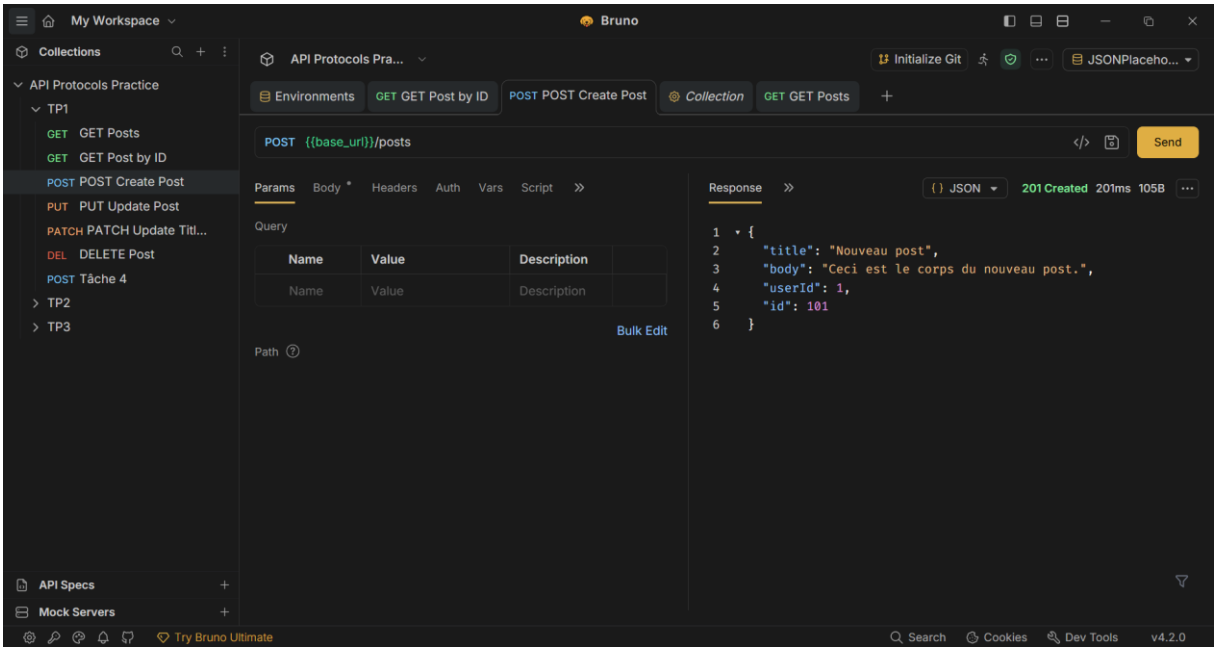


Tache 2 :

- quelle est la différence de rôle entre un en-tête de requête et un en-tête de réponse ? Rappelez, en une phrase, la différence entre Content-Type et Accept (vue en cours).
- *La différence entre ces deux en-têtes est que dans l'en-tête de requête on stipule les données qu'on attend et dans l'en-tête réponse c'est ce que l'API nous répond. Le content-type est le format de donnée que l'on envoie alors que Accept est le format accepté lors de la réponse de l'API*

Tâche 3 :

- quelle différence de principe existe entre PUT et PATCH (rappel du cours : idempotence, remplacement complet vs partiel) ? Le corps que vous envoyez ici illustre-t-il bien cette différence ?
- *La méthode patch va remplacer partiellement un enregistrement tandis que le PUT le remplace entièrement (idempotente). Oui le corps envoyé illustre la différence mais l'API ne trouve pas l'id dans le corps donc crash en erreur 404.*
- si 204 No Content est renvoyé, que contient le corps de la réponse ? Pourquoi ce choix est-il cohérent avec ce qui a été vu en cours ?
- *Le corps de contient rien, car c'est le code de retour classique après un DELETE comme vu en cours.*

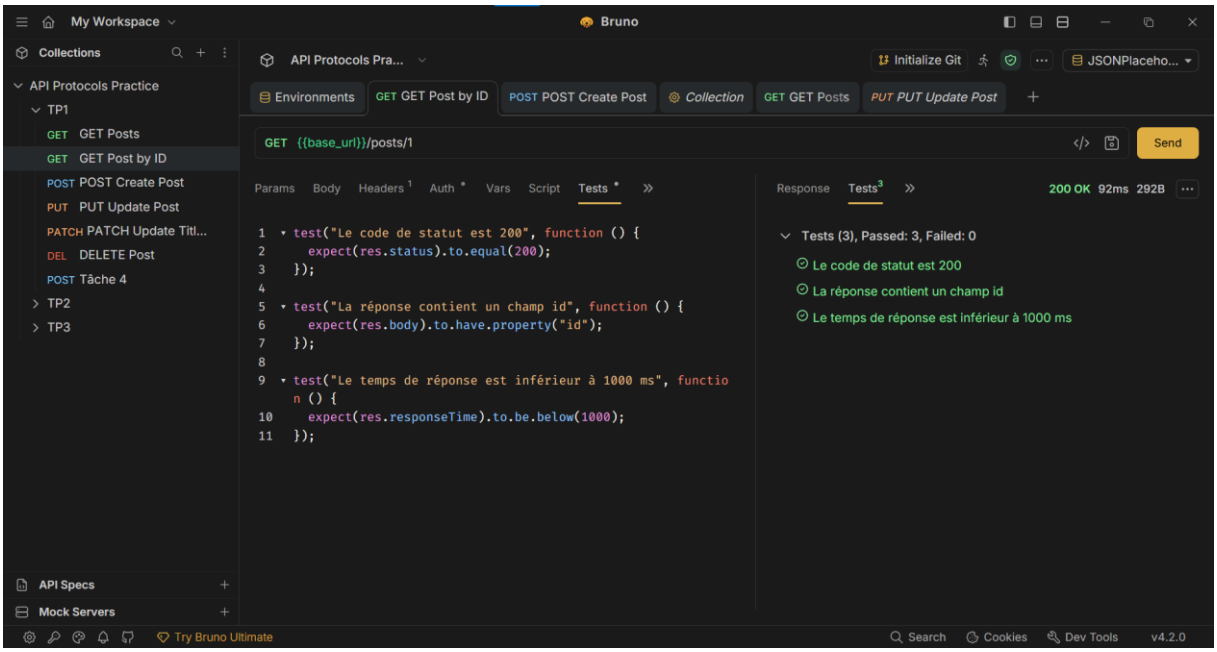


Tâche 4 :

- pour chacun des trois cas, expliquez à qui « appartient » l'erreur (client ou serveur) en vous appuyant sur la classification des codes de statut vue en cours (4xx vs 5xx).
- 404, 405,400 vient du client. Pour le json mal formatté j'ai eu une erreur 500 Internal Server Error qui vient donc du serveur

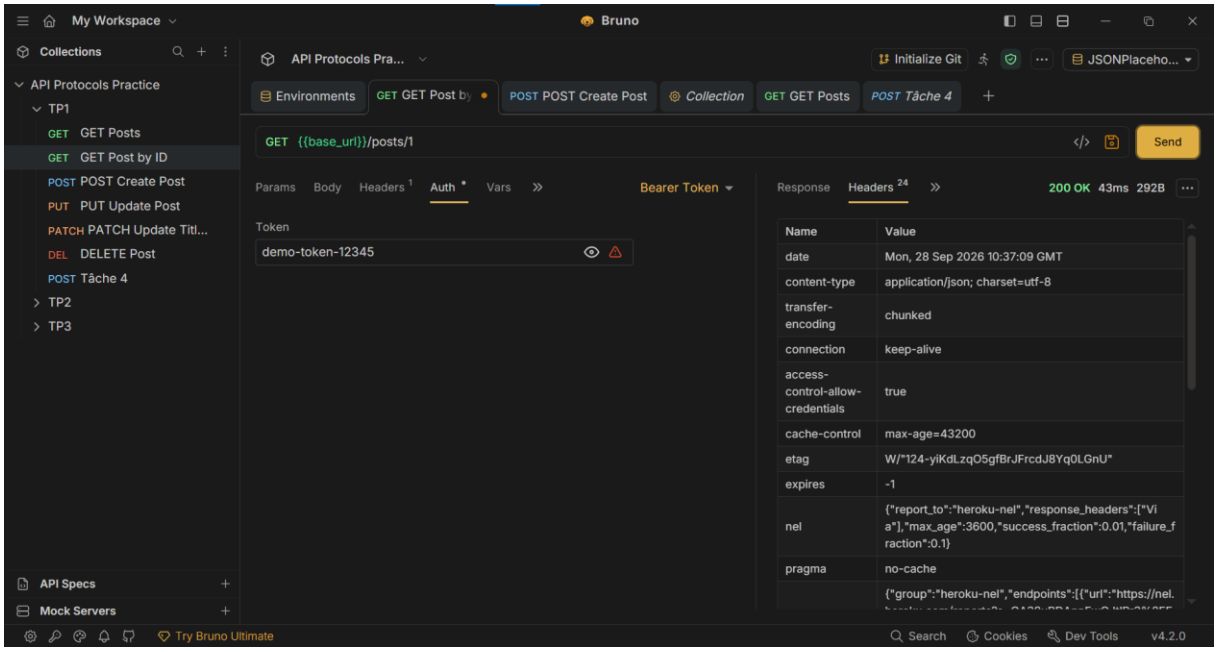
Tâche 5 :

- en quoi ces tests automatisés sont-ils plus fiables qu'une vérification visuelle du corps de la réponse ? Dans quel contexte professionnel (rappel : Chapitre 4, gestion des erreurs) ce mécanisme devient-il indispensable ?
- Ces tests sont plus fiables car ce sont des fonctions natives qui vont faire passer le test si et seulement si la condition est respectée donc pas d'erreur humaine



Tâche 6 :

- Retrouvez, dans l'en-tête généré, la syntaxe exacte vue en cours pour transporter un jeton. Pourquoi ce mécanisme nécessite-t-il HTTPS pour être réellement sûr (rappel du cours) ?
- Car ça passe par une couche chiffrée donc non lisible si interception je n'ai pas le token qui s'affiche.



Tâche 7 :

- En quoi cette documentation générée automatiquement se rapproche-t-elle de la notion de contrat d'API vue au Chapitre 1 ?
- Car il génère une interface qui explique comment utiliser l'API

Tableau :

Tâche	Méthode	URL appelée	Code	Explication
1	GET	{{base_url}}/posts	200	Le serveur renvoie les 100 posts dans un tableau JSON.
1	GET	{{base_url}}/posts?userId=1	200	La collection est filtrée : 10 posts de l'utilisateur 1.
1	GET	{{base_url}}/posts?_page=1&_limit=5	200	La collection est paginée : 5 posts renvoyés.
1	GET	{{base_url}}/posts/1	200	Une ressource unique est renvoyée sous forme d'objet, pas de tableau.
2	GET	{{base_url}}/posts (avec User-Agent et Accept)	200	Même résultat, mais les en-têtes précisent le client et le format attendu.
3	POST	{{base_url}}/posts	201	Le serveur simule la création et attribue l'id 101.
3	PUT	{{base_url}}/posts/1	200	Le post 1 est remplacé en entier par le corps envoyé.
3	PATCH	{{base_url}}/posts/1	200	Seul le titre est modifié, les autres champs sont conservés.
3	DELETE	{{base_url}}/posts/1	200	Suppression simulée, corps {} (204 possible selon le serveur).
4	GET	{{base_url}}/posts/9999	404	Aucune ressource ne correspond à cet identifiant.

Tâche	Méthode	URL appelée	Code	Explication
4	FOO	{{base_url}}/posts/1	à vérifier (405 attendu, souvent 404 ou 500 ici)	La méthode inventée n'est pas supportée par le serveur.
4	POST	{{base_url}}/posts (JSON invalide)	à vérifier (400 attendu)	Le serveur ne peut pas interpréter le corps malformé.
5	POST	{{base_url}}/posts (avec tests)	201	Les assertions sur le statut et sur id passent.
6	GET	{{base_url}}/posts (Bearer token)	200	Le jeton est ajouté dans l'en-tête Authorization, mais le serveur l'ignore.

Réflexion :

HTTP fournit un vocabulaire commun qui rend les API prévisibles. Les méthodes expriment l'intention (lire, créer, remplacer, modifier, supprimer) et leurs propriétés, comme l'idempotence de GET, PUT et DELETE, permettent de relancer une requête sans risque en cas d'échec réseau. Les codes de statut séparent clairement les erreurs du client (4xx) de celles du serveur (5xx), ce qui facilite le diagnostic et la gestion des erreurs. Les en-têtes portent les métadonnées (format, authentification, cache) sans encombrer le corps. Une API qui respecte ces conventions est plus simple à consommer, à déboguer et à faire évoluer.

Les tests automatisés prolongent cette rigueur. Une vérification manuelle est lente, subjective et non reproductible : on regarde une réponse une fois, et on peut passer à côté d'un champ manquant ou d'une lenteur. Une assertion vérifie toujours le même critère, s'exécute en quelques millisecondes et s'intègre dans une chaîne d'intégration continue. Elle transforme le contrat de l'API (statut, structure, performance) en garde-fou, et signale immédiatement toute régression après une modification du serveur.